claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-a62b857a8737424e3.jsonl`
- SHA-256: `fcfebc82e899de6bd810fff2dcb2ae29efb73ae8df9220b3d865f423bc2be891`
- Source modified: `2026-06-18T07:53:30+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_e5140909-890`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T07:52:17.100Z)

```
Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):
  C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.

CANDIDATE:
{
  "title": "load_config crashes at startup if config.json is valid JSON but not an object",
  "file": "backend/config/loader.py",
  "line": "75-86",
  "trigger": "Place a config.json whose top-level JSON is not an object ? e.g. `[]`, `\"x\"`, or
`42`. json.load() at line 69 succeeds (so the try/except at 67-73 does NOT catch it), then `if
file_data:` is truthy and `_unknown_key_paths(file_data, AppConfig)` runs `data.items()` on a
list/str/int -> AttributeError. (Even if that were skipped, `{**_get_builtin_defaults(),
**file_data}` on line 86 also fails for a non-mapping.) The exception is unhandled and
propagates out of load_config, which runs at process startup (backend/main.py::main ->
load_app_config) and via the legacy load_config wrapper, aborting the server/CLI rather than
degrading to defaults.",
  "why_wrong": "The loader's stated contract (docstring lines 60-61) is that a
missing/unreadable file yields a fully-defaulted AppConfig and that bad input is non-fatal. A
syntactically valid but non-object JSON file is parseable yet structurally wrong; it slips past
the only guard (which wraps just the read/parse) and then triggers an unguarded `.items()` /
dict-unpack on untrusted, persisted external data, turning an operator typo into a hard startup
crash with an opaque AttributeError instead of the intended 'use defaults + warn' path.",
  "severity": "low",
  "proposed_fix": "After json.load, coerce non-dict results to the no-op path, e.g. immediately
after line 69 (or before line 75): `if not isinstance(file_data, dict):
logger.warning(f\"{cfg_path} is not a JSON object; ignoring.\"); file_data = {}`. That restores
the documented 'defaults + warn, never fatal' behavior for malformed top-level config.",
  "confidence": 0.8,
  "area": "backend/main.py, backend/infrastructure/database.py, backend/config/, backend/utils/,
backend/providers/ ? API request validation, DB transactions/migrations, config defaults,
hashing, provider error/retry handling."
}

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-
base/backend/config/loader.py + related test. Two lenses below; you are: REACHABILITY ? prove a
real path hits it or set real=false.
```

2. assistant (2026-06-18T07:52:19.453Z)

I'll analyze the candidate finding by reading the actual code and tests.

3. user (2026-06-18T07:52:19.942Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
""") -> list[str]:
26          """Dotted paths in `data` that the typed config will not honor.
27
28          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31          user's intent is lost ? collect every such key so load_config can warn.
32          """
33          unknown: list[str] = []
34          fields = model_cls.model_fields
35          for key, val in data.items():
36              if key not in fields:
37                  unknown.append(prefix + key)
38                  continue
39              ann = fields[key].annotation
40              if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41                  unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
42          return unknown
43
44
45      def _get_builtin_defaults() -> dict[str, Any]:
46          """Return a plain dict of the current built-in defaults.
47
48          The AppConfig model is the single source of truth for defaults.
49          """
50          return AppConfig().model_dump(mode="json")
51
52
53      def load_config(path: str | Path | None = None) -> AppConfig:
54          """Load configuration with the following precedence:
55
56          1. Built-in defaults defined in the Pydantic models.
57          2. Values from the JSON file (if present and readable).
58          3. (Future) environment / CLI overrides.
59
```

```
60          Missing file or unreadable file ? returns a fully defaulted AppConfig.
61          Unknown keys in the file are tolerated (extra="allow" on the model).
62          """
63          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
64
65          file_data: dict[str, Any] = {}
66          if cfg_path.exists():
67              try:
68                  with open(cfg_path, "r", encoding="utf-8") as f:
69                      file_data = json.load(f)
70              except Exception as exc:
71                  # Non-fatal: continue with defaults + whatever we could parse.
72                  # In production we might log here.
73                  logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75          if file_data:
76              unknown = _unknown_key_paths(file_data, AppConfig)
77              if unknown:
78                  logger.warning(
79                      "[CONFIG WARNING] %s contains keys the typed config does not "
80                      "recognize (typo'd or removed knobs ? top-level extras are kept "
81                      "but unread; unknown section keys are silently dropped): %s",
82                      cfg_path, ", ".join(sorted(unknown)),
83                  )
84
85          # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86          merged = {**_get_builtin_defaults(), **file_data}
87
88          return AppConfig.model_validate(merged)
89
90
91      def save_default_config(path: str | Path | None = None) -> Path:
92          """Write a fresh default config.json.
93
94          Used by scripts/doctor.py diagnostics and the --setup CLI path.
95          Overwrites if the file exists.
96          """
97          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98          cfg = AppConfig()
99          cfg_path.parent.mkdir(parents=True, exist_ok=True)
100         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101         return cfg_path
102
103
104     # Backwards-compatible alias used by some older call sites during transition.
105     def get_default_config() -> dict[str, Any]:
106         """Return defaults as a plain dict (for transitional use only)."""
107         return _get_builtin_defaults()
108
```

4. assistant (2026-06-18T07:52:22.461Z)

Now let me trace the reachability ? where `load_config` and `load_app_config` are called,
especially at startup.

5. user (2026-06-18T07:52:25.257Z)

backend\config\loader.py:53:def load_config(path: str | Path | None = None) -> AppConfig:

6. user (2026-06-18T07:52:25.703Z)

backend\config\loader.py:31:   user's intent is lost ? collect every such key so load_config
can warn.

backend\config\loader.py:53:def load_config(path: str | Path | None = None) -> AppConfig:
backend\config__init__.py:4:   from backend.config import load_config, AppConfig, Config
backend\config__init__.py:9:from .loader import load_config, save_default_config
backend\config__init__.py:13:   "load_config",
run_eval.py:26:from backend.config import load_config
run_eval.py:34:   cfg = load_config()
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:159:- Round-trip through
`load_config` + legacy `.get("indexing", {}).get(...)` chains still work.
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:364: -
`test_config_roundtrip_and_typed_reaches_pipeline`: load_config + save_default_config round-
trips; typed AppConfig is used in live paths (e.g. main, validators).
run_regression.py:22:from backend.config import load_config
run_regression.py:28:   cfg = load_config()
docs\BACKLOG.md:100: `backend.config.load_config`. See `CODE_MAP.md` §2.0/§6 (footguns
retired).
docs\CODE_MAP.md:54:
@backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig
docs\CODE_MAP.md:116: - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
docs\CODE_MAP.md:119:- `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths,
interactive `input()` (blocks automation), skips validation. Config via
`backend.config.load_config` (typed); segmenter, padding, and db path all read from the model
(no literal fallbacks).
docs\CODE_MAP.md:120:- `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::_default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
docs\CODE_MAP.md:121:- `@scripts/run_phase3_eval.py::main` ? walk manifest, segment-if-needed,
score, aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id`
(kept as label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the
typed `AppConfig` (same object the live pipeline feeds segmenters).
docs\CODE_MAP.md:122:- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit
codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER
compare (snapshots even a regressing run); `::_default_db_path` resolves via
`backend.config.load_config`.
docs\CODE_MAP.md:125:- `@backend/config/__init__.py` ? ? canonical config import surface. Re-
exports `load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`,
`IndexingConfig`, `StitchingConfig`, `ValidationConfig` (from `.models`); `__all__` = exactly
those 7. Use `from backend.config import load_config, AppConfig`.
docs\CODE_MAP.md:133: - `::load_config` ? precedence built-in-defaults -> file overrides. ?
shallow top-level merge (`{**defaults, **file_data}`): a section in config.json REPLACES the
whole defaults dict for that section (Pydantic then re-applies field defaults for missing
leaves). Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]`
(non-fatal).
docs\CODE_MAP.md:283:- **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/`.indexing` is `None` (the ASGI-import edge case).
docs\CODE_MAP.md:311:| Read config in new code | `from backend.config import load_config,
AppConfig` (`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
docs\CODE_MAP.md:375:| `test_config.py` | `config` (load_config defaults, save_default_config,
AppConfig defaults pinned, `extra` tolerance, `.get()` dict-compat) |
backend\main.py:34:from backend.config import load_config as load_app_config, AppConfig,
StitchingConfig # new typed config
backend\main.py:81:# New code should import load_app_config / AppConfig from backend.config
directly.
backend\main.py:82:def load_config(config_path: str = "config.json") -> Dict[str, Any]:
backend\main.py:83:   cfg = load_app_config(config_path)
backend\main.py:635:   cfg = load_config() # legacy wrapper still works, returns dict for now
backend\main.py:754:   stitching = load_app_config().stitching
backend\main.py:773:   global_config = load_app_config() # returns typed AppConfig
docs\EXPERIMENT_HARNESS.md:134:+ `calibration.leave_one_video_out()` + `metrics.score()`, graded
vs golden (`gt_loader.load_gt_intervals`).

```
tools\generate_highlights.py:38:from backend.config import load_config
tools\generate_highlights.py:91:    cfg = load_config()
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:66:?   ??? loader.py
# load_config, save_default_config, etc.
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:119:-
`backend/config/loader.py` ? `load_config(path=None) -> AppConfig`, `save_default_config(...)`,
internal default merging, `extra="allow"` for transition.
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:120:-
`backend/config/__init__.py` ? Public API re-exports: `from backend.config import load_config,
AppConfig, ...`
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:124:from backend.config
import load_config, AppConfig
docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:125:cfg: AppConfig =
load_config()
scripts\run_phase3_eval.py:33:from backend.config import load_config
scripts\run_phase3_eval.py:91:    cfg = load_config()  # typed AppConfig (same object the live
pipeline feeds segmenters)
scripts\run_process_and_stitch.py:15:from backend.config import load_config
scripts\run_process_and_stitch.py:30:    cfg = load_config()  # typed AppConfig ? single source
of defaults
docs\GOLDEN_REGRESSION_FIXTURES.md:81:Reuses **verbatim**: `metrics.score/interval_iou`,
`experiment.predict/CONTROL/VideoCandidates`, `distill_local.build_candidates`,
`tracknet_runner.parse_trajectory_csv`/`trajectory_to_action_windows`, `rally_gate` net-
crossings (inside `build_candidates`), `gt_loader.load_gt_intervals` (strict),
`regression.gt_hash`, `eval_baselines/` committed-baseline precedent.
tests\test_config.py:14:from backend.config import load_config, save_default_config, AppConfig
tests\test_config.py:19:    cfg =
load_config("/nonexistent/path/that/does/not/exist/12345/config.json")
tests\test_config.py:29:    cfg = load_config("/nonexistent/path/12345/config.json")
tests\test_config.py:35:    cfg = load_config("/nonexistent/path/12345/config.json")
tests\test_config.py:44:        cfg = load_config(p)
tests\test_config.py:53:        cfg = load_config(p)
tests\test_config.py:65:    cfg = load_config("/nonexistent/path/12345/config.json")
tests\test_config.py:82:        cfg = load_config(p)
tests\test_config.py:90:    cfg = load_config(os.path.join(repo_root, "config.json"))
tests\test_config.py:104:        cfg = load_config(p)
tests\test_config.py:129:        cfg = load_config(p)
tests\test_config.py:150:        cfg = load_config(p)
tests\test_config.py:167:        cfg = load_config(str(p))
tests\test_config.py:183:        load_config(str(p))
tests\test_config.py:194:        cfg = load_config(p)
tests\test_config.py:222:        loaded = load_config(p)
tests\test_config.py:236:        cfg = load_config(p)
tests\test_config.py:254:        cfg = load_config(p)
tests\test_config.py:269:    cfg = load_config("/nonexistent/123/config.json")
tests\test_config.py:293:    """Positive for Item 2: save_default_config + load_config round-
trips;
tests\test_config.py:299:        cfg = load_config(p)
docs\QUALITY_ITERATIONS.md:401:Gemini. Config loaded exactly as production (`load_config()`);
segmenters from the registry. Repro:
tests\test_generate_highlights.py:70:    with patch.object(gh, "load_config", return_value={}),
\
tests\test_generate_highlights.py:101:    with patch.object(gh, "load_config",
return_value=cfg), \
tests\test_generate_highlights.py:129:    with patch.object(gh, "load_config",
return_value=cfg), \
```

7. assistant (2026-06-18T07:52:28.285Z)

The candidate is reachable from multiple entry points. Let me verify the startup path in `backend/main.py` and check the existing test for malformed-JSON handling.

8. user (2026-06-18T07:52:29.738Z)

```
760             except Exception as e:
```

```
761                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
762             return
763 
764         if args.debug_clip is not None:
765             if not args.video_path:
766                 print("[ERROR] Please provide --video-path to debug a specific clip.")
767                 return
768             run_cli_diagnose_clip(args.video_path, args.debug_clip)
769             return
770 
771         # Initialize Global Config and DB
772         global global_config, db_instance
773         global_config = load_app_config()  # returns typed AppConfig
774 
775         # The CLI --output-dir overrides the configured output directory. It now lives
776         # on the typed model (OutputConfig.output_dir), so every consumer ? including
777         # /api/compile ? reads the same value instead of a write-only runtime hack.
778         global_config.output.output_dir = args.output_dir
779         os.makedirs(global_config.output.output_dir, exist_ok=True)
780 
781         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782         db_instance = Database(db_path)
783 
784         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785         # the executor is in-process. Mark them failed so pollers see a terminal state.
786         swept = db_instance.fail_stale_jobs()
787         if swept:
788             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789 
```

9. user (2026-06-18T07:52:30.581Z)

```
155     def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
156         """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
157         section keys are silently dropped, so without this warning a typo just makes
158         the knob a no-op (the audit's 'silently dropped config' class)."""
159         p = tmp_path / "typo.json"
160         p.write_text(json.dumps({
161             "some_future_key": 1,                                  # top-level extra
162             "indexing": {"velocty_threshold": 0.5},                # typo'd section key
163             "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
164         }))
165         import logging
166         with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
167             cfg = load_config(str(p))
168         assert isinstance(cfg, AppConfig)  # still non-fatal
169         joined = " ".join(r.message for r in caplog.records)
170         assert "some_future_key" in joined
171         assert "indexing.velocty_threshold" in joined
172         assert "validation.relevance.unknown_gate" in joined
173 
174 
175     def test_recognized_config_does_not_warn(tmp_path, caplog):
176         p = tmp_path / "clean.json"
177         p.write_text(json.dumps({
178             "sport": "badminton",
179             "indexing": {"default_segmenter": "gemini", "wasb": {"velocity_threshold":
0.02}},
180         }))
181         import logging
182         with caplog.at_level(logging.WARNING, logger="backend.config.loader"):
183             load_config(str(p))
184         assert not [r for r in caplog.records if "CONFIG WARNING" in r.message]
```

```
185
186
187     def test_save_default_config_roundtrip():
188         with tempfile.TemporaryDirectory() as td:
189             p = os.path.join(td, "saved.json")
190             saved = save_default_config(p)
191             assert str(saved) == p
192             assert os.path.exists(p)
193
194             cfg = load_config(p)
195             assert cfg.indexing.default_segmenter == "yolo_hybrid"
196             assert cfg.validation.min_blur_threshold == 20.0
197
198
199     def test_model_is_the_source_of_truth():
200         """Changing a default in the model should be reflected without touching json."""
201         # This is a documentation test more than anything.
202         cfg = AppConfig()
203         assert cfg.indexing.tracknet.velocity_threshold == 0.02
204
205
206     def test_output_dir_field_default_and_override():
207         """output_dir now lives on OutputConfig (was a write-only runtime hack), so
208         /api/compile and the CLI resolve the same value. Default + file override + the
209         runtime mutation that backend/main.py::main performs for --output-dir."""
210         cfg = AppConfig()
211         assert cfg.output.output_dir == "output"  # built-in default
212
213         # The CLI override path: main() assigns args.output_dir onto the typed model.
214         cfg.output.output_dir = "/tmp/custom_out"
215         assert cfg.output.output_dir == "/tmp/custom_out"
216
217         # File override is honored on load.
218         with tempfile.TemporaryDirectory() as td:
219             p = os.path.join(td, "out_override.json")
220             with open(p, "w") as f:
221                 json.dump({"output": {"output_dir": "reels", "db_name": "x.db"}}, f)
222             loaded = load_config(p)
223             assert loaded.output.output_dir == "reels"
224             assert loaded.output.db_name == "x.db"
225
226     def test_config_shallow_section_replacement():
227         """Positive test for Item 2 (config loader): shallow section replacement.
228
229         File-provided 'indexing' section replaces the defaults 'indexing' section entirely
```

10. assistant (2026-06-18T07:52:33.611Z)

Now let me look at the test around line 167-183 ? the "malformed JSON" test (`test_config.py:167` and `:183`), and check the area near lines 100-150 for any non-object JSON test.

11. user (2026-06-18T07:52:34.949Z)

```
95
96      def test_fusion_config_file_override():
97          with tempfile.TemporaryDirectory() as td:
98              p = os.path.join(td, "fusion.json")
99              with open(p, "w") as fh:
100                 json.dump({"indexing": {"fusion": {
101                     "substrate": "tracknet", "min_crossings": 2,
102                     "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1],
[0.5, 0.9]],
103                 }}}, fh)
104             cfg = load_config(p)
```

```
            f = cfg.indexing.fusion
            assert f.substrate == "tracknet"
            assert f.min_crossings == 2
            assert f.cue_flags == {"person": True}
            assert len(f.court_polygon) == 3
            # An old config without a fusion block still loads (defaults applied).
            assert cfg.indexing.wasb.velocity_threshold == 0.02


    def test_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "override.json")
            data = {
                "indexing": {
                    "default_segmenter": "gemini",
                    "ai_max_workers": 3,
                },
                "validation": {
                    "min_blur_threshold": 42.0,
                },
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "gemini"
            assert cfg.indexing.ai_max_workers == 3
            assert cfg.validation.min_blur_threshold == 42.0
            # Unspecified keys fall back to defaults
            assert cfg.sport == "badminton"


    def test_extra_keys_tolerated():
        """Old or future keys in config.json must not break loading."""
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "with_extra.json")
            data = {
                "sport": "badminton",
                "indexing": {"default_segmenter": "yolo_hybrid"},
                "some_future_key": "ignored_for_now",
                "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.relevance.court_pixels_min_ratio == 0.1


```

12. assistant (2026-06-18T07:52:37.339Z)

Let me check the malformed-JSON test at lines 167-183 to see what the existing test asserts.

13. user (2026-06-18T07:52:38.519Z)

```
    def test_unknown_keys_warn_with_dotted_paths(tmp_path, caplog):
        """Typo'd keys must be LOUD: top-level extras are kept-but-unread and unknown
        section keys are silently dropped, so without this warning a typo just makes
        the knob a no-op (the audit's 'silently dropped config' class)."""
        p = tmp_path / "typo.json"
        p.write_text(json.dumps({
            "some_future_key": 1,                                # top-level extra
            "indexing": {"velocty_threshold": 0.5},              # typo'd section key
```

```
163             "validation": {"relevance": {"unknown_gate": True}},   # doubly nested
164         }))
```

14. assistant (2026-06-18T07:52:41.361Z)

Let me find the test at line 167-183 region precisely, which contained a `load_config(str(p))` that did NOT assign to cfg (line 183 raw call) ? possibly a "raises" test.

15. user (2026-06-18T07:52:42.199Z)

```
36          assert cfg.indexing.skip_ai_handoff is False
37
38
39      def test_skip_ai_handoff_file_override():
40          with tempfile.TemporaryDirectory() as td:
41              p = os.path.join(td, "skip.json")
42              with open(p, "w") as fh:
43                  json.dump({"indexing": {"skip_ai_handoff": True}}, fh)
44              cfg = load_config(p)
45              assert cfg.indexing.skip_ai_handoff is True
46
47
48      def test_logging_config_file_override():
49          with tempfile.TemporaryDirectory() as td:
50              p = os.path.join(td, "logging.json")
51              with open(p, "w") as fh:
52                  json.dump({"logging": {"verbose_http": True}}, fh)
53              cfg = load_config(p)
54              assert cfg.logging.verbose_http is True
55
56
57      def test_fusion_config_defaults_reproduce_control():
58          """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
mask.
59
60          Gate A (min_crossings) defaults to 0 = OFF: the #146 served-default flip was
REVERTED
61          (eval?serve ? the +0.074 was on the harness's recall-oriented PROPOSAL windowing; on
the
62          served COARSE windowing Gate A is neutral/negative, ?0.008, dropping real rallies).
See
63          docs/QUALITY_ITERATIONS.md and the served litmus in
tests/test_served_gate_a_regression.py.
64          """
65          cfg = load_config("/nonexistent/path/12345/config.json")
66          f = cfg.indexing.fusion
67          assert f.substrate == "wasb"
68          assert f.cue_flags == {}            # person cue OFF
69          assert f.min_crossings == 0         # Gate A OFF (default) ? #146 served flip
reverted
70          assert f.min_players == 0           # Gate B OFF
71          assert f.court_polygon is None      # no mask
72          assert f.velocity_threshold == 0.02
73
74
75      def test_fusion_config_gate_a_opt_in_via_min_crossings():
76          """Gate A is opt-in: setting min_crossings explicitly enables it (the eval-only
operating
77          point), while the served default stays OFF (#146 flip reverted)."""
78          with tempfile.TemporaryDirectory() as td:
79              p = os.path.join(td, "optin.json")
80              with open(p, "w") as fh:
81                  json.dump({"indexing": {"fusion": {"min_crossings": 2}}}, fh)
82              cfg = load_config(p)
83              assert cfg.indexing.fusion.min_crossings == 2   # Gate A ON only when explicitly
```

```
set
84
85
86      def test_served_default_segmenter_is_gemini():
87          """The shipped config.json served default is the prior detector (gemini) ? the #146
flip
88          to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
89          repo_root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
90          cfg = load_config(os.path.join(repo_root, "config.json"))
91          assert cfg.indexing.default_segmenter == "gemini"
92          # ...and the served fusion default keeps Gate A OFF (no served gating).
93          assert cfg.indexing.fusion.min_crossings == 0
94
95
```

16. assistant (2026-06-18T07:52:45.606Z)

Now let me look at the test region around lines 100-130 and 167-184 in full to confirm there is no existing test for non-object top-level JSON (the malformed-JSON test).

17. user (2026-06-18T07:52:47.488Z)

```
271:    assert isinstance(validation, dict)
274:    assert isinstance(validation.get("relevance"), dict)
```

18. assistant (2026-06-18T07:52:51.609Z)

No existing test feeds a non-object top-level JSON. Let me confirm the malformed-bytes test at line 167 to see whether it covers invalid JSON (which `json.load` catches) vs. valid-but-non-object JSON.

19. user (2026-06-18T07:52:52.482Z)

```
227         """Positive test for Item 2 (config loader): shallow section replacement.
228
229         File-provided 'indexing' section replaces the defaults 'indexing' section entirely
230         (no deep merge of sub-keys), per loader's {**defaults, **file_data}.
231         """
232         with tempfile.TemporaryDirectory() as td:
233             p = os.path.join(td, "shallow.json")
234             with open(p, "w") as fh:
235                 json.dump({"indexing": {"default_segmenter": "motion"}}, fh)
236             cfg = load_config(p)
237             assert cfg.indexing.default_segmenter == "motion"
238             # Pydantic defaults fill the rest of the (replaced) section
239             assert cfg.indexing.ai_max_workers == 5
240
241
242     def test_config_unknown_key_paths_warning_and_tolerate():
243         """Positive test for Item 2: _unknown_key_paths warning for unknown top-level keys,
244         but tolerated (extra='allow'), and unknown section keys dropped silently.
245         """
246         with tempfile.TemporaryDirectory() as td:
247             p = os.path.join(td, "unknown.json")
248             with open(p, "w") as fh:
249                 json.dump({
250                     "indexing": {"default_segmenter": "motion"},
251                     "completely_unknown_top": "bar",
252                     "validation": {"unknown_in_section": 99}  # dropped
253                 }, fh)
254             cfg = load_config(p)
255             assert cfg.indexing.default_segmenter == "motion"
256             # tolerated, no crash
257
```

```
258
259     def test_get_returns_dict_for_nested_sections_legacy_compat():
260         """Regression: legacy consumers (validators/segmenters) do
261         ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
262         .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
263         chained access raises 'X object has no attribute get' on every real run.
264
265         This is the original strong legacy test (restored per #160 review). It covers the
266         full set of footgun guards: dict sections, nested dicts, scalar leaves with
defaults,
267         missing key fallbacks, and leaf keys discoverable inside sections.
268         """
269         cfg = load_config("/nonexistent/123/config.json")
270         validation = cfg.get("validation", {})
271         assert isinstance(validation, dict)
```

20. assistant (2026-06-18T07:52:57.056Z)

I've confirmed the full test file has no test feeding a non-object top-level JSON (`[]`, `42`, `"x"`). The candidate's reachability claim is solid. Let me verify the trigger concretely by reproducing the failure in an isolated Python process (the dev container has pydantic for config).

21. user (2026-06-18T07:53:00.186Z)

```
payload='[]'     -> CRASH TypeError: 'list' object is not a mapping
payload='"x"'    -> CRASH AttributeError: 'str' object has no attribute 'items'
payload='42'     -> CRASH AttributeError: 'int' object has no attribute 'items'
```

22. assistant (2026-06-18T07:53:12.898Z)

Reproduced exactly. All three non-object payloads crash unhandled:
- `[]` ? `TypeError: 'list' object is not a mapping` (at line 86 `{**defaults, **file_data}`; `_unknown_key_paths` actually does NOT crash on a list because `for key, val in data.items()` ? wait, a list has no `.items()`, so it crashes at line 35 first). Let me note the precise crash point.

For `[]`: `_unknown_key_paths` calls `data.items()` on a list at line 35 ? `AttributeError`... but the repro shows `TypeError: 'list' object is not a mapping`. That means `if file_data:` for `[]` is falsy (empty list), so the guard at line 75 is skipped, and it crashes at line 86 dict-unpack instead. For `"x"` and `42` (truthy), it crashes at line 35 inside `_unknown_key_paths` via `.items()`. The candidate correctly identified both crash sites.

Now the REACHABILITY lens. The trigger requires a `config.json` whose top-level JSON is a valid non-object (array/string/number). This is reached from real, non-test entry points:

- `backend/main.py::main` (line 773) calls `load_app_config()` (the `load_config` alias) at process startup with the default `config.json` path ? server/CLI startup.
- `run_eval.py:34`, `run_regression.py:28`, `scripts/run_phase3_eval.py:91`, `scripts/run_process_and_stitch.py:30`, `tools/generate_highlights.py:91` all call `load_config()` with the default path.

The input is operator-controlled persisted external data (a `config.json` file on disk). An operator typo such as writing a top-level JSON array or accidentally saving a bare value produces a hard crash with an opaque error, instead of the documented "missing/unreadable file ? fully defaulted AppConfig + warn, never fatal" behavior (docstring lines 60-61, and the `except Exception` at 70-73 explicitly intends non-fatal degradation).

The path is genuinely reachable: real production entry points load the default `config.json`, and the only guard wraps the read/parse (which succeeds for valid-but-non-object JSON), so the malformed structure slips through to an unguarded `.items()` / dict-unpack. No existing test or guard covers the non-object top-level case (confirmed by reading the entire `test_config.py`).

I'll now report the REACHABILITY verdict.

23. user (2026-06-18T07:53:22.474Z)

Structured output provided successfully

24. assistant (2026-06-18T07:53:30.217Z)

Reported. The candidate is a real, reachable defect ? verdict: real=true, reachable=true, not already guarded/tested, not intended.

Key facts confirmed:
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/config/loader.py` lines 75?86: non-object top-level JSON crashes unhandled. Empty list `[]` is falsy ? skips the line-75 guard ? crashes at line 86 (`{**_get_builtin_defaults(), **file_data}`, `TypeError: 'list' object is not a mapping`). Truthy non-objects (`"x"`, `42`) crash at line 35 inside `_unknown_key_paths` (`AttributeError: ...has no attribute 'items'`).
- Reproduced all three crashes in-process.
- Reachable at real startup: `backend/main.py:773` (`load_app_config()`), plus `run_eval.py:34`, `run_regression.py:28`, `scripts/run_phase3_eval.py:91`, `scripts/run_process_and_stitch.py:30`, `tools/generate_highlights.py:91` ? all loading the default `config.json`.
- The only guard (`try/except` at lines 67?73) wraps only the read+`json.load`, which succeeds for valid-but-non-object JSON.
- Contract violated: docstring (lines 60?61) and the `except` branch intend a non-fatal "defaults + warn" path.
- No test in `tests/test_config.py` covers a non-object top-level JSON (full file read).